

13.2 Histórico e auditoria de mudanças — status de entrega

Bloco 13.2 Histórico e auditoria de mudanças · Sprint 13 — Administração, qualidade e segurança · **adiantado** a pedido do Breno em 2026-09-23 · Data: 2026-09-23 · Ambiente: gc-dev (gzbmhgnpoehoirmnidmgg)

Resumo

Nasceu de um pedido: uma opção de **logs e auditoria no menu lateral**, com filtro de automação e de outras operações, que registre tudo o que é feito no sistema, com os erros e os sucessos. O ClickUp já tinha isso como o 13.2, e a parte da automação cruza com a tarefa 4. Tratar os casos que dão errado da lista OCR. O Breno decidiu três coisas: adiantar o 13.2, esperar o 5.7 fechar e deixar a tela **só para admin**.

Esta é a **etapa A** de três: banco, trigger e tela. Toda escrita que chega ao banco, em 15 tabelas, vira um evento com quem, quando, de onde e o diff campo a campo. A tela /logs lista, filtra e abre o diff. A RPC que registra **erro** também já existe e está testada, mas ninguém a chama ainda: ligar as Server Actions a ela é a etapa B, e ligar o n8n é a etapa C.

1. As cinco atividades do card

ATIVIDADE DO CLICKUP	ESTADO
Decidir a abordagem: jsonb por entidade ou tabela central	Tabela central auditoria_eventos . O historico jsonb de propostas e orçamentos continua, porque alimenta a aba Histórico e guarda o motivo de rejeição
Registrar quem, quando, campo, valor anterior e valor novo	✓ autor_id / autor_descricao , em , detalhe.campos.<coluna>.{de, para}
Aba Histórico funcional em orçamento, proposta, contrato e NF	Parcial. Orçamento e proposta já tinham (4.6 e migration 013). Contrato e NF não têm tela ainda (sprints 6 e 9); quando tiverem, a aba lê desta tabela por entidade + registro_id
Registrar as mudanças de status de todas as entidades	✓ Update que muda status vira acao = 'status' , nas 15 tabelas
Trigger de auditoria no banco para as tabelas financeiras	✓ contratos , notas_fiscais , pagamentos , acordos_pagamento , acordo_parcelas e fd estão entre as 15

2. O que foi entregue

2.1 Banco: migrations 019 e 020, aplicadas em gc-dev

supabase/migrations/20260923130000_auditoria_eventos.sql (019):

- **auditoria_eventos**: empresa_id, em, origem (sistema / automacao / banco), entidade, registro_id, acao, resultado (sucesso / erro), mensagem, autor_id, autor_descricao e detalhe jsonb. Índices em (empresa_id, em desc) e (entidade, registro_id).
- **Trigger auditar_mudanca**, after insert or update or delete, nas 15 tabelas com empresa_id: clientes, orçamentos, obras, propostas, contratos, itens, execução, notas fiscais, pagamentos, acordos, parcelas, fd, documentos da automação, contatos da automação e profiles.
- **RPC registrar_evento**, para o que o trigger não vê.
- **RLS**: só admin lê, e só da própria empresa. Não há policy de escrita.
- **Imutável**: update é recusado por trigger (auditoria_imutavel) para qualquer papel, inclusive a chave de serviço.

supabase/migrations/20260923131000_auditoria_referencia.sql (020): coluna **referencia** (número da proposta, código da obra, nome do cliente; em itens, a descrição), gravada pelo trigger, e p_referencia na RPC. Ver decisão 5.

migration list : 20/20 alinhadas.

2.2 Tela /logs e menu

- Item "Logs e auditoria" no menu lateral (src/lib/nav.ts), só para admin.
- src/app/(app)/logs/: layout.tsx com requirePerfil(['admin']), page.tsx, filters.tsx e eventos-table.tsx, no padrão de /propostas: estado todo na URL, busca com debounce de 300 ms, PAGE_SIZE = 20, Pagination, período por computePeriodoCutoff.
- **Filtros**: origem, resultado, entidade, ação, autor e período. A busca procura na referência, na mensagem e no autor; colar um uuid procura o registro.
- **Colunas**: quando (fuso de São Paulo), origem, resultado, entidade + referência (com link para o detalhe em proposta, orçamento, obra e FD), ação, autor e resumo. O resumo abre um <details> com o diff (antes → depois) no update, ou com a linha inteira na criação e na exclusão.
- formatDate novo em src/lib/format.ts.

2.3 Helper puro src/lib/auditoria.ts

Rótulos, opções dos filtros, validação da querystring, o .or() da busca, a leitura tolerante do detalhe (a RPC aceita qualquer jsonb), o resumo da linha e o link do registro. Sem use client e sem React; tem 10 casos em src/lib/auditoria.test.ts.

2.4 Seed `supabase/seed_auditoria.sql`

Dois eventos fixos para as camadas runtime, dados e navegador, já que a camada de escrita limpa o que gera: um **erro da automação** (`SEED-AUDITORIA-ERRO`) e uma **mudança de status com diff** apontando para `SEED-VENCIDA-001` . É idempotente.

3. Decisões registradas

1. **A origem sai de quem está conectado, não de parâmetro.** Com `auth.uid()` é `sistema` ; com a chave de serviço (n8n, rota de ingestão) é `automacao` ; SQL direto como postgres (seed, migration) é `banco` . Assim ninguém consegue registrar um evento como se fosse outra origem.
2. **Auditoria nunca derruba a gravação de negócio.** O corpo do trigger está num bloco `exception when others` que vira `WARNING` . Perder um evento é ruim; perder a proposta que a ingestão gravava porque o log falhou é pior.
3. **Autor sem sessão.** Na **criação**, vale o `created_by` da linha: é o profile de serviço que a ingestão grava, pela decisão 12 do plano do Telegram. No update e no delete sem sessão, `created_by` é o autor **original**, então fica só `autor_descricao` (`service_role`).
4. **Diff sem ruído.** `updated_at` e `historico` ficam fora do diff (o status já entra pela própria coluna), e `dados_extraidos` aparece como `[omitido]` . Update que só mexe nessas colunas não gera evento.
5. **referencia gravada no evento, não lida na hora de mostrar.** Sem ela, a tela não respondia à pergunta mais comum ("o que aconteceu com a P-123?"). Ela fica gravada porque o registro pode ter sido excluído, e é justamente na exclusão que a auditoria mais importa.
6. **A RPC com sessão ignora `p_empresa_id`** . Quem tem sessão só registra na própria empresa e como si mesmo. Sem sessão, só a chave de serviço chama, e ela precisa informar a empresa. O `anon` não tem `execute` .
7. **Delete só pela chave de serviço.** Não há policy; é o que a limpeza do plano de validação usa e o que uma rotina de retenção usaria.

4. Números reais do `npm run validar`

Rodado duas vezes em 2026-09-23 contra gc-dev, com `VALIDACAO_PORTA=3211`
`VALIDACAO_PORTA_CDP=9322` (ver seção 6):

- **isolado:** o 13.2 sozinho, sobre o fechamento da sprint 5 (`7fc592d`), comparado com a baseline do `docs/sprint-5/5.8-status-entrega.md` ;
- **merge:** o 13.2 aplicado sobre o `5b9f76f` (sprint 5 + Fase 6 da automação, PR #6), que é exatamente o que está no working tree. A Fase 6 **sem** o 13.2 não foi medida por esta sessão, e por isso a coluna não tem a diferença.

CAMADA	FIM DA SPRINT 5	13.2 ISOLADO	13.2 + FASE 6 (MERGE)
estático	limpo	limpo	<code>tsc --noEmit</code> limpo, ☒ No ESLint warnings or errors
unitário	146 casos	156 casos (+10 de <code>auditoria.test.ts</code>)	177 casos, 0 falhas
build	32 rotas	33 rotas (<code>/logs</code>)	34 rotas, iguais à baseline (<code>/logs</code> + <code>/api/ingestao/proposta</code>)
runtime	47/47 rotas	57/57	57/57 rotas ok
dados	18/18 checagens	21/21	21/21 checagens ok
escrita	122/122 passos	136/136	160/160 passos ok
navegador	57/57 passos	62/62	62/62 passos ok , console limpo

As 10 rotas novas do runtime cobrem:

- `/logs` com filtros;
- a busca pelos dois seeds;
- o filtro de erro;
- a busca pelo número da proposta;
- o resultado vazio;
- o item de menu presente para admin e ausente para comercial;
- o 307 para comercial, visualizador e financeiro.

As 3 checagens novas de dados são a query da page, os autores e a contraprova de RLS (o comercial vê 0). Os 5 passos novos do navegador são o menu, a busca pela tela, a exclusão registrada, o filtro de erro e o `<details>` com o diff.

4.1 Os passos novos da camada de escrita (14 no isolado, 15 no merge)

- `createProposta` gerou `criar` com autor, origem `sistema`, referência e a linha.
- rascunho → enviada virou `status` com o diff.
- Nenhum evento de update traz só `updated_at` / `historico`.
- A busca pelo número da proposta acha os eventos dela.
- As 2 transições do orçamento viraram 2 eventos `status`.
- `createItem` gerou `criar` em itens.
- `deleteProposta` deixou `excluir` com a linha apagada.
- O comercial registra erro pela RPC; o evento sai com origem `sistema`, autor comercial e resultado `erro`.

- A RPC com sessão ignora `p_empresa_id`.
- O comercial grava mas não lê o próprio evento.
- Nem a chave de serviço altera um evento.
- `anon` não chama `registrar_evento`.
- **Só no merge:** a proposta criada pela rota de ingestão entra como origem `automacao`, com o profile de serviço como autor. É o caso real do filtro "Automação".
- A limpeza apagou **1203 eventos** gerados pelo roteiro no merge (1189 no isolado).

Screenshots da última rodada em `/tmp/gc-validacao/shots-13.2/`: `15-logs-busca.png`, `16-logs-diff.png` e `17-logs-390px.png` são desta tela.

5. O que NÃO foi validado — pendências nominais

1. **Erros das Server Actions não são gravados ainda (etapa B).** A RPC existe e foi exercitada, mas nenhuma das 39 Server Actions a chama. Hoje a tela só mostra **erro** vindo do seed. O filtro "Erro" funciona; o que falta é o dado.
2. **O n8n não grava ainda (etapa C).** O `Processar Documento` e o Error Trigger do `Notificar` continuam avisando só o grupo de admin do Telegram. As escritas que o n8n faz com a chave de serviço **já aparecem** como `automacao`, pelo trigger. Os erros dele, não.
3. **Conferência visual não foi feita.** As screenshots existem, mas ninguém olhou; a camada navegador prova texto e comportamento, não aparência. O mesmo vale para a largura de 390px: a tabela tem rolagem horizontal própria, e isso não foi medido.
4. **O caminho do `WARNING` (auditoria falhou, gravação seguiu) não foi exercitado.** A decisão 2 está no código e não tem teste: provocar a falha exigiria quebrar a tabela de propósito.
5. **Scripts com chave de serviço aparecem como `automacao`.** O `seed-propostas-dev.mjs` e afins caem na mesma origem que o n8n. Em gc-dev isso é ruído aceitável; em prod só o n8n e a ingestão usam a chave.
6. **Qualquer perfil com sessão pode registrar evento pela RPC.** Precisa ser assim para a etapa B, e o evento sai sempre com o autor verdadeiro. Mas um usuário pode encher o log com eventos em nome próprio. Não há limite de taxa.
7. **Sem retenção.** A tabela só cresce. Uma rodada do plano gera por volta de 1.300 eventos e os limpa; o uso real não tem rotina de expurgo. Fica para quando houver volume medido.
8. **empresas não é auditada** (não tem `empresa_id` e só muda por migration).
9. **Aba Histórico de contrato e NF:** depende das telas das sprints 6 e 9.

6. Como este bloco foi feito

O trabalho foi feito em **worktrees destacados, sem branch e sem commit**, porque outra sessão escrevia a Fase 6 da automação no mesmo checkout ao mesmo tempo, com um `validar.sh build --aceitar-rotas` rodando. Dois `next build` na mesma `.next` e duas edições no mesmo script se atropelam.

1. `.claude/worktrees/13-2-logs` , a partir do `7fc592d` : desenvolvimento e a rodada "isolado".
2. Quando a Fase 6 foi mesclada (`5b9f76f` , PR #6), o diff foi aplicado em `.claude/worktrees/13-2-merge` , a partir do `5b9f76f` . Houve **um conflito**, o esperado, em `scripts/validar-escrita.mjs` : as duas frentes acrescentavam variáveis no topo e uma seção antes do `finally` . Foi resolvido mantendo as duas, com a Fase 6 primeiro, e ganhou o passo que liga uma à outra (a proposta da ingestão aparece como `automacao`). A baseline de rotas entrou limpa, com 34. O `types.ts` regenerado ficou idêntico ao do patch. Foi daqui a rodada "merge".
3. O resultado do merge foi aplicado no **working tree** do checkout principal, sem commit, como pede o `CLAUDE.md` . O build não foi rodado lá, porque havia um `next dev` no ar naquele checkout. O conteúdo é o mesmo que passou na rodada "merge".

Registro para quem for operar gc-prod

Migrations pendentes em gc-prod, nesta ordem, depois das do 5.7 e da Fase 6:

`20260923130000_auditoria_eventos.sql` e `20260923131000_auditoria_referencia.sql` . Só criam coisa nova (tabela, funções, triggers) e não mexem em dado existente; o trigger começa a registrar a partir da aplicação. O seed `seed_auditoria.sql` é de dev e **não** vai para prod.